

# Технологии Программирования

Тёмная магия



# Вопрос!

Что есть RAI?

# Хранители мира

## RAII (*Resource Acquisition Is Initialization*)

- Переменная объявляется непосредственно перед использованием,
- Переменная удаляется после использования,
- Нельзя переиспользовать переменные.

## Типы

- Использовать максимально говорящие типы, минимально необходимого размера.  
`uint8_t m_drawing_context;` //из типа видно, что контекстов не может быть больше 255
- Для сложных типов предпочитать структуры с именованными полями, стараться минимизировать использование безымянных таплов или пар.

## Имя - существительное

- Имя должно быть кратким и содержать информацию о том, что храниться в переменной  
`uint16_t m_color_rgb565;` //переменная содержит 3х компонентный цвет размером 2 байта в раскладке r5, g6, b5



**ВСЕГДА ИНИЦИАЛИЗИРУЙТЕ ПЕРЕМЕННЫЕ СРАЗУ: В МОМЕНТ ОБЪЯВЛЕНИЯ**

# Бессмертный

## Время Жизни Переменных

- Кроме имени переменной и ее значения важен контроль времени жизни переменной.
- При работе с памятью время жизни особенно важно, т.к. объекты созданные на куче (operator new) не удаляются при выходе из области видимости указателя.
- При потере указателя на объект его память остается недоступной для записи, и не может быть переиспользована (memory leak).
- При большом количестве утечек, программа может заблокировать всю доступную память.
- Решение:
  - **Автоматизировать процесс работы с памятью**

```
bool doSomeStuff(int param, int flag)
{
    int* dynamicArray = new int [param];
    if (param > 100)
    {
        delete [] dynamicArray;
        return true;
    }
    int i = 0;
    for (; i < flag; ++i)
    {
        if (i == param)
        {
            delete [] dynamicArray; //codedup
            return true;
        }
    }
    if (i % 2)
    {
        delete [] dynamicArray; //codedup
        throw "something strange happened";
    }
    do
    {
        if (0 == param)
        {
            delete [] dynamicArray; //codedup
            throw "div by zero";
        }
        flag /= param;
        flag -= param;
    } while (flag >= 0);
    delete [] dynamicArray; //codedup
    return false;
}
```

# Основные руны

Умные указатели автоматически следят за временем жизни переменной выделенной на куче.



## Unique pointer.

- Указатель с уникальным владением объектом в памяти,
- Не может быть присвоен,
- Не может быть скопирован,
- При удалении объекта типа `unique_ptr` удаляется объект на который он указывает,
- По скорости работы сравним с обычным указателем.

## Shared pointer.

- Указатель с разделяемым владением объектом в памяти,
- Следит за количеством копий указателя на объект, содержит счетчик `reference`
- При удалении объекта типа `shared_ptr` уменьшает количество `references` на 1,
- Если количество `references` равно 0, объект в памяти удаляется.

## Weak pointer.

- Указатель без владения объектом в памяти, слабое владение,
- Следит за количеством слабых указателей,
- Может сконструировать `shared_ptr` из себя - `lock()`,
- Может указывать на удаленный объект, но позволяет проверить валидность содержимого - `expired()`,
- Конструируется только из `shared_ptr`.

# Вопрос!

Что еще кроме объектов в памяти может быть забыто и не освобождено программой?

- Файлы,
- Потоки,
- Объекты синхронизации потоков,
- ...

Для таких объектов аналогов smart pointers нет.

**Или есть?**



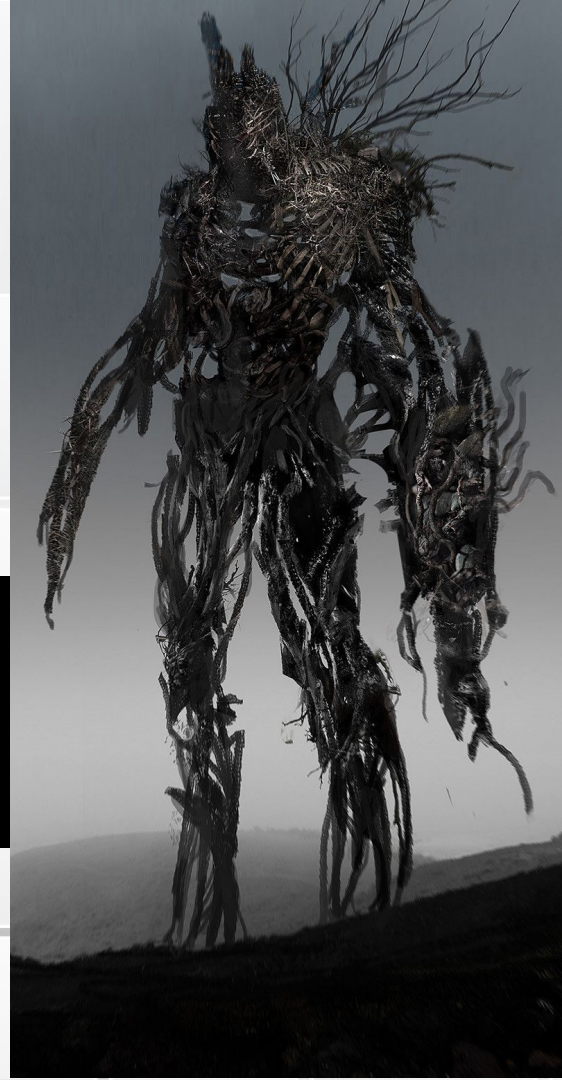
# Страж

## Guard classes (RAII classes)

- Классы обеспечивающие правильный цикл работы своего содержимого вне зависимости от условий (кроме abnormal termination),
- Используют концепцию конструкторов/деструкторов для обеспечения контроля за временем жизни охраняемого объекта,
- Должны быть реализованы на стеке, в куче такие классы не работают,

```
class MutexGuard {  
    std::mutex& m_mutex;  
public:  
    MutexGuard(std::mutex& mutex) : m_mutex(mutex) {}  
    void Lock() {m_mutex.lock();};  
    // освобождает мьютекс при выходе из области видимости Гарда, чем бы это не было вызвано  
    ~MutexGuard() {m_mutex.unlock();};  
}
```

- Есть стандартные объекты для объектов синхронизации в STL:  
std::lock\_guard
- Для других объектов guard классы нужно писать самим.



# Черная Белена

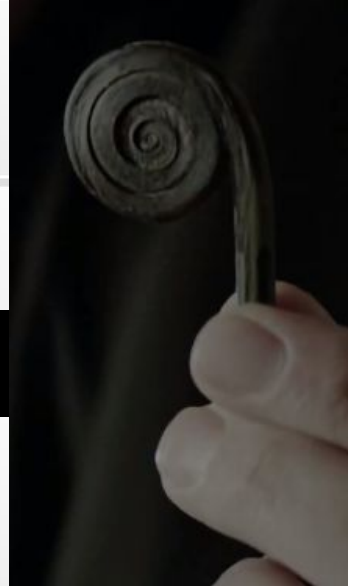
## Шаблон

```
UniversalMultifieldStruct<uint32_t, float, 4> templateStructure = {10, 2.f, {}};
```

- Инструкция для компилятора по интерпретации типов данных,
- Шаблонный тип выводится (определяется) в момент использования, а не написания,
- Каждый вывод шаблона с отличающимися параметрами создает новый тип в вашей программе,
- Шаблоны существуют для типов,
- Шаблоны существуют для функции.

```
std::array<uint32_t, 16> array{0}; //uint32_t - тип параметров в массиве, 16 - его размер
```

```
uint32_t c_array[16] = {0}; // эквивалент, однако с ним не будут работать функции STL
```

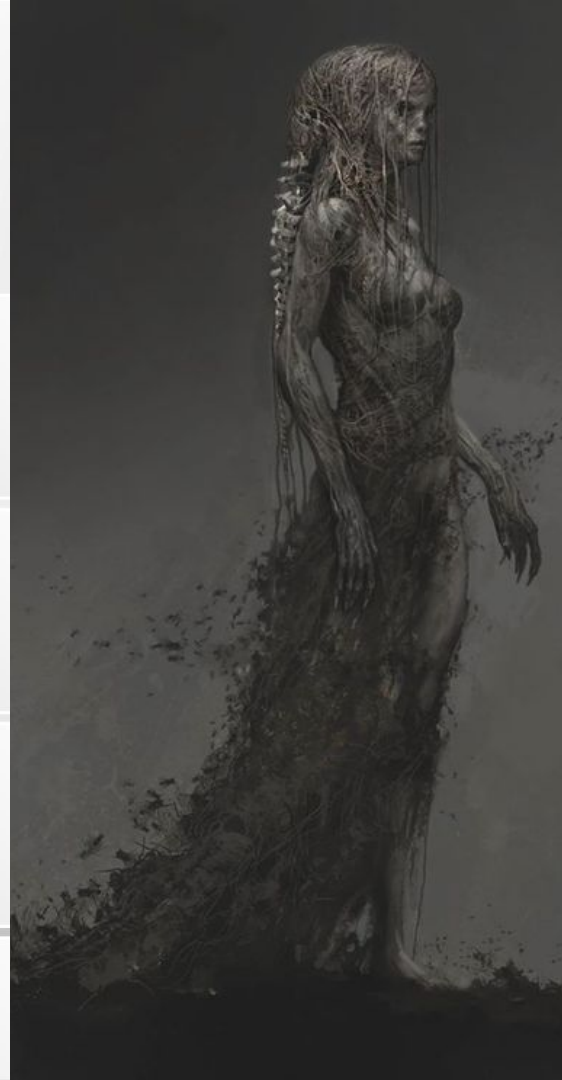




# Королева темных магов

## SFINAE “Substitution Failure Is Not An Error”

- Связано всегда с перегрузкой функций и шаблонами.
- Правило применяется при автоматическом выводе типов шаблона по аргументам функции.
- Правило определяет какой шаблон отбросить в случае его несоответствия к текущей операции, при наличии более подходящего кандидата.
- Шаблонные перегрузки могут быть отброшены (проигнорированы) в том случае, когда их инстанцирование приведет к синтаксической ошибке,
- SFINAE рассматривает только заголовок функции, ошибки в теле функции не будут пропущены.



# Сердце Королевы

```
// Явно определенная функция
int difference(int val1, int val2)
{
    return val1 - val2;
}
```

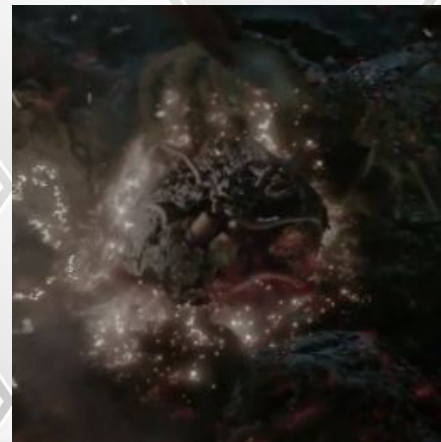
```
// Шаблон
template<typename T>
typename T::difference_type difference(const T& val1, const T& val2)
{
    return val1 - val2;
}
```

- Компилятор сконструирует шаблонный заголовок для функции:

```
int::difference_type difference(int val1, int val2) // Синтаксическая ошибка
```

- Такой шаблон не может быть подставлен в виду синтаксической ошибки.
- Даже в случае правильности конструкции, функция с явным указанием типа будет иметь больший приоритет, поэтому подстановочный шаблон будет отброшен после сравнения,
- SFINAE требует чтобы данная конструкция была отброшена, без сообщения программисту об ошибке.
- Ошибка по SFINAE будет сгенерирована, если подходящего шаблона нет или их больше одного (компилятор выбирать не умеет).

**В данном случае мы никак не влияем на то, как шаблон будет выведен**



# Вопрос!

А может ли программист повлиять на то, как выводится шаблон?

**Compile-time analysis и type\_traits**



## static\_assert

- Статический ассерт - инструкция компилятору, сгенерировать ошибку, если условие на этапе компиляции ложно.
- Используется для формирования пользовательской ошибки компиляции в случаях, когда синтаксис не нарушен, и, в целом, код соответствует стандарту, но нарушает логику программы.

```
static_assert(sizeof(int) == 8, "x86 architecture is not supported for this algorithm, IA64 is required");
```

- Статический ассерт - инструкция компилятору, проверить параметры в данной точке.

```
⊗ static_assert failed: 'x86 architecture is not supported for this algorithm, IA64 is required' [E:\Development\Education\build\templates\templates.vcxproj] MSVC(C2338)
```

- Можно использовать в шаблонах, для проверки параметров шаблонов, чтобы предотвратить их неверное использование.

# Кто ты?



## Type\_traits

- Контроль типа передаваемого в шаблон на этапе компиляции. Библиотека STL содержит набор мета функций позволяющих контролировать поведение шаблонов в зависимости от параметров.
- Варианты типов C++
  - Тривиальные `std::is_trivial`
  - Интегральные `std::is_integral`
  - Указатели `std::is_pointer`
  - Пустые `std::is_empty`
  - ...
- Шаблоны можно специфицировать в зависимости от типов передаваемых параметров или блокировать компиляцию в случае неверных данных шаблона

```
template<typename T>
typename T difference(const T& val1, const T& val2) {
    static_assert(std::is_arithmetic<T>::value); //Вызовет ошибку компиляции, если тип не является числом
    return val1 - val2;
}
```

# Темные заветы



## Метаметоды

- Кроме проверки на тип параметра, существует набор операций с `type_traits`.  
Позволяющими более точно настроить проверку типов или инстанцирование шаблона.
  - Сравнение типов
  - Условная инициализация
  - Проверка условий во время компиляции
  - Выведение обобщенного типа
  - ...
- С помощью этих функций можно более точно специфицировать шаблон или даже запретить использование шаблона для определенных типов, константных параметров и их комбинаций.

```
// Шаблон будет работать только для арифметических типов
template<typename T, typename std::enable_if<std::is_arithmetic<T>::value>::type = true>
typename T difference(const T& val1, const T& val2) {
    return val1 - val2;
}
```



# Твоя заклятие

```
template<typename T>
typename T difference(const T& val1, const T& val2) {
    return val1 - val2;
}
```

- Шаблон будет работать неправильно
  - для беззнаковых типов,
  - для указателей,
  - для классов...

```
template<typename T, typename std::enable_if<std::is_signed<T>::value>::type = true>
typename T difference(const T& val1, const T& val2) {
    return val1 - val2;
}
```

```
template<typename T, typename std::enable_if<std::is_signed<T>::value>::type = false>
typename T difference(const T& val1, const T& val2) {
    return val1 > val2 ? val1 - val2 : 0;
} // Шаблон будет инстанцироваться не только для беззнаковых но и для всех остальных: указателей, классов
```

```
template<typename T>
typename T difference(const T& val1, const T& val2) {
    static_assert(!std::is_arithmetic<T>::value, "для данного типа функция не существует");
}
```

A decorative pattern of light gray hexagons with thin white outlines, arranged in a staggered grid and fading out towards the right side of the slide.

**THANK YOU!**